Verilog HDL

Dr. E.Papanasam
papanasam.e@vit.ac.in
AP Senior
School of Electronics
VIT Chennai

Importance of HDLs

- HDLs have many advantages compared to traditional schematic-based design
- Designs can be described at a very abstract level by use of HDLs
- Designers can write their RTL description without choosing a specific fabrication technology
- Logic synthesis tools can automatically convert the design to any fabrication technology.
- If a new technology emerges, designers do not need to redesign their circuit
- They simply input the RTL description to the logic synthesis tool and create a new gate-level netlist, using the new fabrication technology.

Introduction

Digital IC's are classified as

- Small Scale Integration Circuit (SSI)
- Medium Scale Integration (MSI)
- Large Scale Integration (LSI)
- Very Large Scale Integration (VLSI) (more than 1, 00, 000 transistors).
- Ultra Large Scale Integration (ULSI)

- Smaller circuits can be verified using simple Breadboard connections.
- Highly impossible with LSI and higher integrated circuits.
- As the design becomes more and more difficult Computer Aided Design (CAD/EDA) are evolved for the design and testing of Digital circuits.
- So CAD tools are more popular.

Introduction

- For a long time, programming languages such as FORTRAN, Pascal, and C were being used to describe computer programs that were sequential in nature.
- Similarly, in the digital design field, designers felt the need for a standard language to describe digital circuits. Thus, Hardware Description Languages (HDLs) came into existence.
- Hardware description languages such as Verilog HDL and VHDL became popular.
- **Verilog (VERIfiable LOGic) HDL originated in 1983** at Gateway Design Automation. Later, **VHDL** was developed under contract from DARPA.
- Both **Verilog and VHDL** simulators simulate large digital circuits quickly gained acceptance from designers.

Introduction to Verilog HDL

- Verilog Simulators:
  - Commercial Simulators:
  - Xilinx ISE
  - Active HDL
  - Model Sim
  - Quartus-II
  - MP Sim
- Online Simulators:
  - iverilog.com
  - EDAPlayground.com
  - easyeda.com

- Open Source Simulators:
  - Verilator
  - Icarus Verilog
  - GPL cver
  - VeriWell

Lexical Conventions- White spaces and Comments

Whitespace:

- Blank space (\b)
- Tab (\t)
- Newline (\n)

Comments

```
a = b && c; // This is a one-line comment
/* This is a multiple line comment */
/* This is /* an illegal */ comment */
```

Operators

- Operators are of three types: unary, binary, and ternary.
- Unary operators precede the operand.

  a = ~ b; // ~ is a unary operator. b is the operand
- Binary operators appear between two operands
- a = b && c; // && is a binary operator. b and c are operands
- Ternary operators have two separate operators that separate three operands.
- a = b ? c : d; // ?: is a ternary operator. b, c and d are operands

Lexical Conventions-Numbers

- Number Specification allowed are
  - Sized numbers
  - Unsized numbers
  - Unknown and high-impedance values
  - Negative numbers
- Underscore character and question marks used to improve readability.
- Not allowed as the first character.
- Examples:
  - 12’b1111_0000_1010
  - – ‘?’ is the same as ‘z’ (only regarding numbers)
  - 4’b10?? // the same as 4’b10zz

Lexical Conventions-Sized Numbers

- Size of Number is specified along with the number.
- General syntax: **<size>'<base><number>**

  <size> number of bits in number (in decimal).

  <number> is the number in radix.

  '<base> type of number

  - 'd or D for decimal (radix 10)
  - 'b or B for binary (radix 2)
  - 'o or O for octal (radix 8)
  - 'h or H for hexadecimal (radix 16)

Examples:

4'b1111, 12'habc, 8'd255

Lexical Conventions-Unsized Numbers

- Size of Number is not specified. ***Default size is at least 32*** (depends on Verilog compiler).
- **Default base is decimal.**
- Examples:
  - 23232
  - ’habc
  - ’o234
- Negative numbers can be used with **negative sign (-) before size**.
- Examples:
  - -6’d35
  - 4’d-12 // illegal

Lexical Conventions-Unsized Numbers

X or Z values:

- **Unknown value**: lowercase x or Uppercase X
  - 4 bits in hex, 3 bits in octal, 1 bit in binary.
- **High-impedance value**: lowercase z or Uppercase Z
  - 4 bits in hex, 3 bits in octal, 1 bit in binary.
- Examples:
  - 12’h13x
  - 4’hX
  - 32’bz

Data Types

- The Data types used in Verilog are
  - Value Set
  - Nets
  - Registers
  - Vectors
  - Integer, Real, and Time Registers
  - Arrays
  - Memories
  - Parameters
  - Strings

Data Types

- Value Set:
  - Verilog Supports 4 values and 8 strengths.

S. No.	Value Level	Condition in Hardware Circuits
1	0	Logic zero, false condition
2	1	Logic one, true condition
3	X	Unknown value
4	Z	High impedance, floating state

Data Types

- Value Set:
  - Strength levels are often used to resolve conflicts between drivers of different strengths in digital circuits.
  - Value levels 0 and 1 can have the strength levels.

S. No.	Strength level	Type	Degree
1	Supply	Driving	Strongest
2	Strong	Driving	↑
3	Pull	Driving	
4	Large	Storage	
5	Weak	Driving	
6	Medium	Storage	
7	Small	Storage	
8	Highz	High Impedance	Weakest

Data Types

- If two signals of unequal strengths are driven on a wire, **the stronger signal prevails.**
- For example, if two signals of strength strong1 and weak 0 contend, the result is resolved as a strong 1.
- If **two signals of equal strengths** are driven on a wire, **the result is unknown**.
- If two signals of strength **strong 1 and strong 0 conflict**, the result is an **x**.
- Strength levels are particularly useful for accurate modelling of signal contention, MOS devices, dynamic MOS, and other low-level devices.
- Only *trireg* nets can have storage strengths large, medium, and small.

Data Types-Nets

- Nets represent **connections between hardware elements.**
- Just as in real circuits, **nets have values continuously driven** on them by the outputs of devices that they are connected to.
- Nets are declared primarily with the keyword **wire**.
- Nets are one-bit values by default unless they are declared explicitly as vectors.
- The terms **wire and net** are often used interchangeable.
- The default value of a net is **z** (except the trireg net, which defaults to x).
- Nets get the output value of their drivers.
- If a net has no driver, it gets the value z.

Data Types-Nets

- net is not a keyword but represents a class of data types such as wire, wand, wor, tri, triand, trior, trireg, etc.
- The **wire declaration is used most frequently**.

```
wire a;                    / / Declare net a for the circuit
wire b, c;   / / Declare two wires b, c for the circuit
wire d = 1'b0;             / / Net d is fixed to logic value 0 at declaration.
```

Data Types-Registers

- Registers represent **data storage elements.**
- Registers retain value until another value is placed onto them.
- In Verilog, the term **register merely means a variable that can hold a value**.
- Unlike a net, **a register does not need a driver**.
- Verilog registers do not need a clock as hardware registers do.
- Values of registers can be changed anytime in a simulation by assigning a new value to the register.
- Register data types are commonly declared by the keyword **reg**.
- The default value for a reg data type is x.

Data Types-Registers

- Registers represent **data storage elements.**
- Registers retain value until another value is placed onto them.

```
reg reset; / / declare a variable reset that can hold its value
initial                    / / this construct will be discussed later
begin
reset = 1'b1;              //initialize reset to 1 to reset the digital circuit.
#100  reset = 1'b0;        // after 100 time units reset is deasserted.
end
```

Data Types-Vectors

- **Nets or reg** data types can be declared as vectors (multiple bit widths).
- If bit width is not specified, the default is scalar (l-bit).
- Vectors can be declared at [high#:low#] or [low# :high#], but the left number in the squared brackets is always the most significant bit of the vector.

```
wire a;                          / / scalar net variable, default.
wire [7:0] bus;                  / / 8-bit bus.
wire [31:0] busA, busB, busC;    / / 3 buses of 32-bit width.
reg clock;                       / / scalar register, default.
reg [0:40] virtual-addr;         //vector register, virtual address 41 bits wide.
```

Data Types-Vectors

- Examples: wire b; //scalar net variable
  wire[7:0] databus; //8-bit data bus
- Represent buses
  - wire[3:0] busA;
  - reg [1:4] busB;
  - reg [1:0] busC;
- Left number is always MSB.
- Vector assignment **by position** also possible.
  - busB[1] = busA[3];
  - busB[2] = busA[2];
  - busB[3] = busA[1];
  - busB[4] = busA[0];

Data Types-Integer

- An integer is a **general purpose register** data type used for manipulating quantities.
- Integers are declared by the keyword **integer.**
- Although it is possible to use **reg as a general-purpose variable**, it is more convenient to declare an integer variable for purposes such as counting.
- The default width for an integer is the host-machine word size, which is implementation specific but is at least 32 bits.
- Registers declared as data type reg store values as unsigned quantities, whereas integers store values as signed quantities.

```
integer counter;        / / general purpose variable used as a counter.
initial
counter = -1;           / / A negative one is stored in the counter
```

Data Types-Real

- Real number constants and real register data types are declared with the keyword **real.**
- They can be specified in decimal notation (e.g., 3.14) or in scientific notation (e.g., 3e6, which is 3×10^6).
- Real numbers **cannot have a range declaration**, and their **default value is 0.**
- When a real value is assigned to an integer, the real number is rounded off to the nearest integer.

Data Types-Real

- Real number constants and real register data types are declared with the keyword **real**.

```
real delta;                 / / Define a real variable called delta
initial
begin
            delta = 4e10;           / / delta is assigned in scientific notation
            delta = 2.13;           / / delta is assigned a value 2.13
end
integer i;                  / / Define an integer i
initial
i = delta;                  / / i gets the value 2 (rounded value of 2.13)
```

Data Types-Time

- Time register is **used to store and manipulate time values**.
- They are used to store simulation time and it is declared with the keyword **time**.
- Syntax:

```
Time time_id1, time_id2,……..time_idn
```

Examples:

```
time currtime;
initial
        begin
                currtime=$time;
end
```

Data Types-Arrays

- Arrays are allowed in Verilog for reg, integer, time, and vector register data types.
- Arrays are not allowed for real variables.
- Arrays are accessed by <array-name> [<subscript>].
- Multidimensional arrays are not permitted in Verilog.
- It is important not to confuse arrays with net or register vectors.
- A vector is a single element that is n-bits wide.
- On the other hand, arrays are multiple elements that are l-bit or n-bits wide.

Data Types-Arrays

- Arrays are allowed in Verilog for reg, integer, time, and vector register data types.

```
integer count[0:7];          //  An array of 8 count variables
reg booll[31:01;             //Array of 32 one-bit Boolean register variables
time chk_point[1:100];       //Array of 100 time checkpoint variables
reg [ 4:0] port_id[0 :7] ;   //Array of 8 port_ids; each port_id is 5 bits wide
integer matrix[4:0][4:0];    // Illegal declaration. Multidimensional array
count[5]                     // 5th  element of array of count variables
chk_point[100]               //100th  time check point value
Port_id[3]                   //3rd  element of port-id array. This is a 5-bit value.
```

Data Types-Memories

- In digital simulation, one often needs to model **register files, RAMS, and ROMs.**
- Memories are modelled in Verilog simply as **an array of registers**.
- Each element of the array is known as **a word**.
- Each word can be one or more bits. It is important to differentiate between n 1-bit registers and one n-bit register.
- A particular word in memory is obtained by using the address as a memory array subscript.

```
reg mem1bit[0:1023];          // Memory mem1bit with 1K 1-bit words
reg [ 7:0] membyte [0:1023];  // Memory membyte with 1K 8-bit words (bytes)
```

Data Types-parameters

- Verilog allows constants to be defined in a module by the keyword parameter.
- Parameters cannot be used as variables.
- Parameter values for each module instance can be overridden individually at compile time.

```
parameter port_id = 5;                    // Defines a constant port-id
parameter cache_line_width=256;           //Constant defines width of cache line.
parameter bussize = 8;
Parameter bit=1, byte=8;
reg [bussize-1 : 0] databus;
```

Data Types-Strings

- Strings can be stored in **reg**.
- The width of the register variables must be large enough to hold the string.
- Each character in the string takes up 8 bits (1 byte).
- If the width of the register is greater than the size of the string, **Verilog fills bits to the left of the string with zeros.**
- If the register width is smaller than the string width, **Verilog truncates the leftmost bits of the string.**
- It is always safe to declare **a string that is slightly wider than necessary**.

```
reg [8*18:1] string-value;      //Declare a variable that is 18 bytes wide
initial
string-value = "Hello Verilog World";        // String can be stored in variable
```

Data Types-Strings

- Some special characters are used along with the strings for the purpose of display.

S. No.	Escaped Characters	Character Displayed
1	\n	New line
2	\t	Tab
3	%%	%
4	\\	\
5	\”	“
6	\000	Character written in 1-3 octal digits

Data Types-Usage restrictions

Data Flow and Structural Modeling

- Can use only **wire** data type.
- Cannot use **reg** data type.

Behavioral Modeling

- Can use only **reg** data type (within initial and always constructs).
- Cannot use **wire** data type.

Operators

Verilog Operator	Name	Functional Group
[ ]	bit-select or part-select	
( )	parenthesis	
!	logical negation	logical
~	negation	bit-wise
&	reduction AND	reduction
|	reduction OR	reduction
~&	reduction NAND	reduction
~|	reduction NOR	reduction
^	reduction XOR	reduction
~^ or ^~	reduction XNOR	reduction
+	unary (sign) plus	arithmetic
-	unary (sign) minus	arithmetic
{ }	concatenation	concatenation
{{ }}	replication	replication
*	multiply	arithmetic
/	divide	arithmetic
%	modulus	arithmetic

Operators

<< >>	shift left shift right	shift shift
>>>= << <=	greater than greater than or equal to less than less than or equal to	relational relational relational relational
== !=	logical equality logical inequality	equality equality
=== !==	case equality case inequality	equality equality
&	bit-wise AND	bit-wise
^ ^~ or ~^	bit-wise XOR bit-wise XNOR	bit-wise bit-wise
|	bit-wise OR	bit-wise
&&	logical AND	logical
||	logical OR	logical
?:	conditional	conditional

Logical Operators

The Logical Operators are

&& →**logical AND**

|| →**logical OR**

! →**logical NOT**

- Operands evaluated to ONE bit value: 0, 1 or x.
- Result is ONE bit value: 0, 1 or x.
- <u>Example:</u>

A = 1; B = 0; C = x;

A && B // → 1 && 0 →0

A || !B // → 1 || 1 → 1

C || B // →x || 0 →x

Equality Operators

The Equality Operators are

==	→ logical equality	Returns 0, 1, or X.
!=	→logical inequality	
===	→ case equality	Returns 0, or 1.
!==	→case inequality	

```
// A = 4, B = 3
// X = 4'b1010, Y = 4'b1101
// Z = 4'b1xxz, M = 4'b1xxz, N = 4'b1xxx

A == B // Results in logical 0
X != Y // Results in logical 1
X == Z // Results in x
Z === M //Results in logical 1 (all bits match, including x and z)
Z === N //Results in logical 0 (least significant bit does not match)
M !== N // Results in logical 1
```

Bitwise Operators

The Bitwise Operators operates over operand bit by bit.

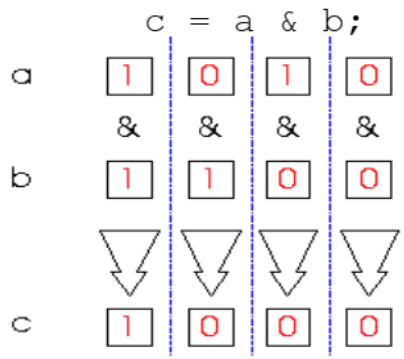
& → **bitwise AND**

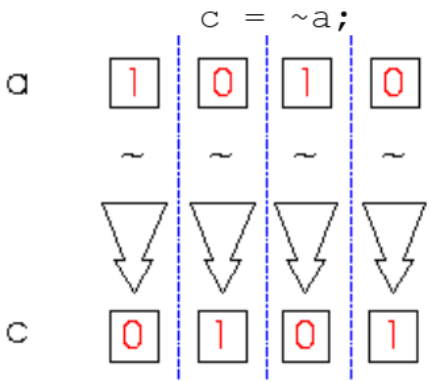
| → **bitwise OR**

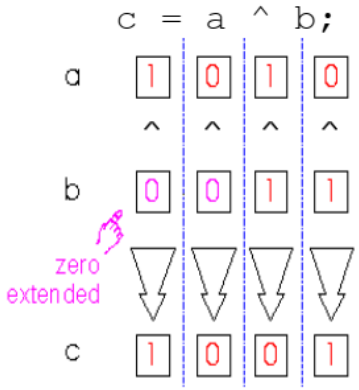
~ → **bitwise NOT**

^ → **bitwise XOR**

~^ or ^~ → **bitwise XNOR**







Reduction Operators

The reduction operators operates over single operand and results in a single bit of result.

& → **AND**

| → **OR**

^ → **XOR**

~& →**NAND**

~| →**NOR**

~^ or ^~ →**XNOR**

- One multi-bit operand →One single-bit result.

<u>Example:</u>

```
a = 4'b1001;
c = |a;          // c = 1|0|0|1 = 1
```

Shift Operators

Used to left shift/ right shift the data. (circular).

>> →**shift right**

<< →**shift left**

>>> →**arithmetic shift right**

<<< →**arithmetic shift left**

Example:

- Result is same size as first operand, always zero filled

  a = 4’b1010; d = a >> 2; // d = 0010

  c = a << 1; // c = 0100

- Vacant position filled by either LSB (>>>) or MSB (<<<) value

  a = 4’b0101; d = a >>>1; // d = 1010

  c = a <<<1; // c = 1010

Concatenation Operators

Used to combine two or more numbers/ variables.

{op1, op2, ..} →concatenates op1, op2, .. to single number

- Operands must be sized !!

<u>Example:</u>

```
reg a;
reg [2:0] b, c;
a = 1'b 1;
b = 3'b 010;
c = 3'b 101;
catx = {a, b, c};            // catx = 1_010_101
caty = {b, 2'b11, a};        // caty = 010_11_1
catz = {b, 1}; // WRONG !!
```

Replication ..

```
catr = {4{a}, b, 2{c}}; // catr = 1111_010_101101
```

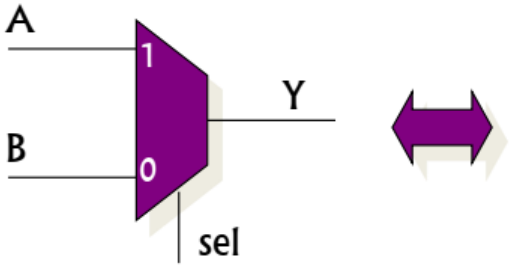
Conditional Operators

The conditional operators operates as

cond_expr ? true_expr : false_expr.

Example:

- A 2 X 1 MUX can be expressed as



Y = (sel)? A : B;

Operators Precedence

Highest	~	NOT
	*, /, %	mult, div, mod
	+, -	add,sub
	<<, >>	shift
	<<<, >>>	arithmetic shift
	<, <=, >, >=	comparison
	==, !=	equal, not equal
	&, ~&	AND, NAND
	^, ~^	XOR, XNOR
	|, ~|	OR, NOR
Lowest	?:	ternary operator

System Tasks

- Verilog provides standard system tasks to do certain routine operations.
- All system tasks appear in the form $<keyword>.
- Operations such as displaying on the screen, monitoring values of nets, stopping, and finishing are done by system tasks.
- Displaying information:
- $display is the main system task for displaying values of variables or strings or expressions.
- Very similar to printf in c

```
//Display the string in quotes
$display("HelloVerilog World");

-- Hello Verilog World
```

```
//Display value of current simulation time 230
$display($time);

-- 230
```

System Tasks

- Some other systems tasks are
- Verilog provides a mechanism to monitor a signal when its value changes. This facility is provided by the $monitor task.

```
//Monitor time and value of the signals clock and reset
//Clock toggles every 5 time units and reset goes down at 10 time units
initial
begin
    $monitor($time,
                " Value of signals clock = %b reset = %b", clock,reset);
end
```

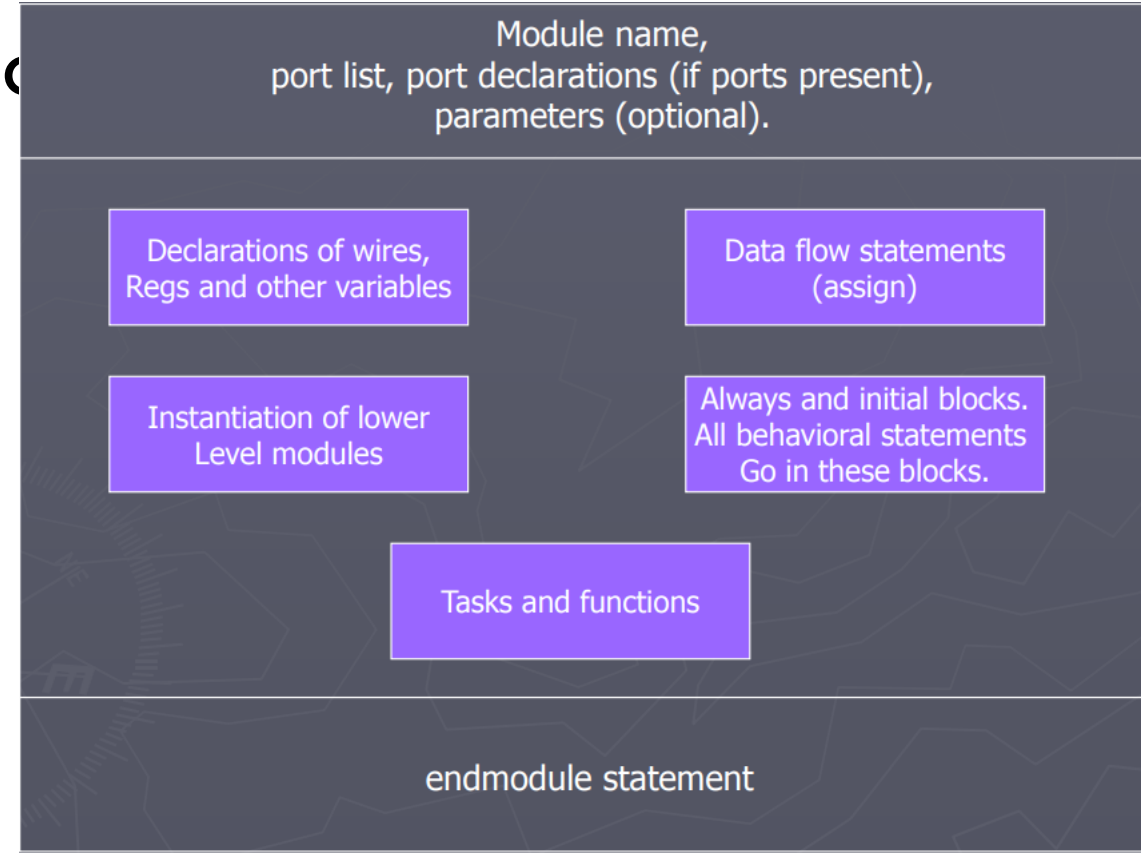
Partial output of the monitor statement:

```
-- 0 Value of signals clock = 0 reset = 1
-- 5 Value of signals clock = 1 reset = 1
-- 10 Value of signals clock = 0 reset = 0
```

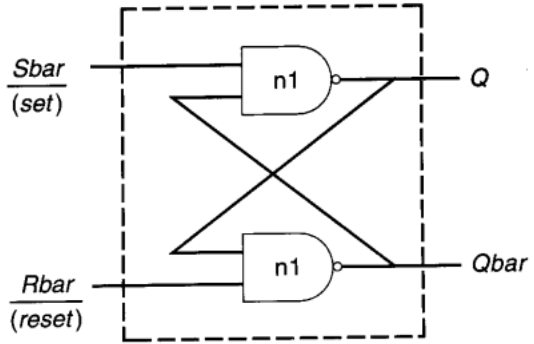
- The task $stop is provided to stop during a simulation.
- The $finish task terminates the simulation.

Modules and Ports

- Co



Example

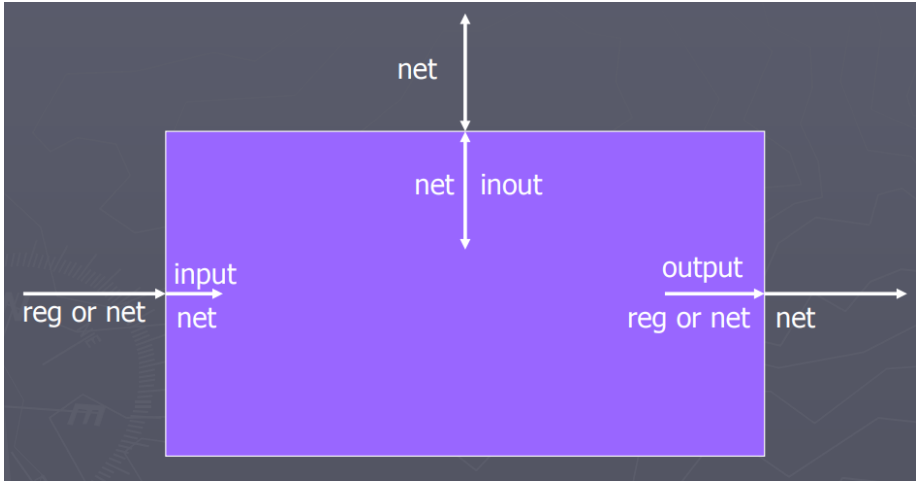



```
// SR_latch module
module SR_latch(Q, Qbar, Sbar, Rbar);

//Port declarations
output Q, Qbar;
input Sbar, Rbar;

// Instantiate lower-level modules
// In this case, instantiate Verilog primitive nand gates
// Note, how the wires are connected in a cross-coupled fashion.
nand n1(Q, Sbar, Qbar);
nand n2(Qbar, Rbar, Q);

// endmodule statement
endmodule
```

Port

Verilog Keyword	Type of Port
`input`	Input port
`output`	Output port
`inout`	Bidirectional port

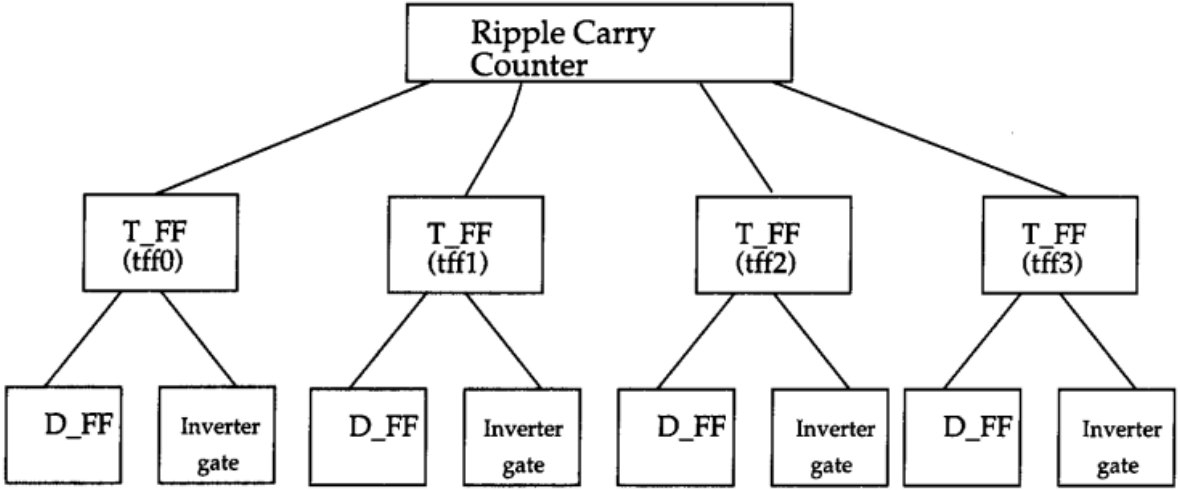
- Port connection rules
- Port consisting of two units: internal to the module. Another external to the module



Module Instantiation

- A module provides a template from which one can create actual objects
- When a module is invoked, Verilog creates a unique object from the template
- Each object has its own name, variables, parameters, and I/O interface
- The process of creating objects from a module template is called instantiation
- The objects are called instances

Module Instantiation Example



```
module ripple_carry_counter(q, clk, reset);
output [3:0] q;
input clk, reset;
T_FF tff0(q[0],clk, reset);
T_FF tff1(q[1],q[0], reset);
T_FF tff2(q[2],q[1], reset);
T_FF tff3(q[3],q[2], reset);
endmodule

module T_FF(q, clk, reset);
output q;
input clk, reset;
wire d;
D_FF dff0(q, d, clk, reset); // Instantiate D_FF. Call it dff0.
not n1(d, q); // not gate is a Verilog primitive.
endmodule
```

```
module D_FF(q, d, clk, reset);
output q;
input d, clk, reset;
reg q;
always @(posedge reset or negedge clk)
if (reset)
    q = 1'b0;
else
    q = d;
endmodule
```

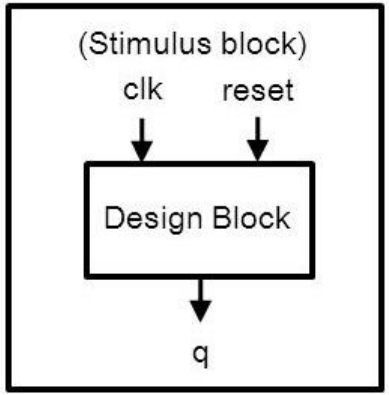
Testbench

- Once a design block is completed, **it must be tested**.
- The functionality of the design block can be tested by applying stimulus and checking results.
- We call such a **block the stimulus block also called as test bench.** It is good practice to keep the stimulus and design blocks separate.
- The stimulus block can be written in Verilog. A separate language is not required to describe stimulus.
- Different test benches can be used to thoroughly test the design block.
- Two styles of stimulus application are possible.
  - Stimulus Block Instantiates Design Block
  - Stimulus and Design Blocks instantiated in a Dummy Top-Level Module

Testbench

1. Stimulus Block Instantiates Design Block:

- In this simulation method, the stimulus block instantiates the design block and directly drives the signals in the design block.
- Here, the stimulus block becomes the top-level block.
- It manipulates input signals, and it checks and displays the output signal.



Example

```
module stimulus;

reg clk;
reg reset;
wire[3:0] q;

// instantiate the design block
ripple_carry_counter r1(q, clk, reset);

// Control the clk signal that drives the design block. Cycle time = 10
initial
    clk = 1'b0; //set clk to 0
always
    #5 clk = ~clk; //toggle clk every 5 time units

// Control the reset signal that drives the design block
// reset is asserted from 0 to 20 and from 200 to 220.
initial
begin
    reset = 1'b1;
    #15 reset = 1'b0;
    #180 reset = 1'b1;
    #10 reset = 1'b0;
    #20 $finish; //terminate the simulation
end

// Monitor the outputs
initial
    $monitor($time, " Output q = %d",  q);

endmodule
```

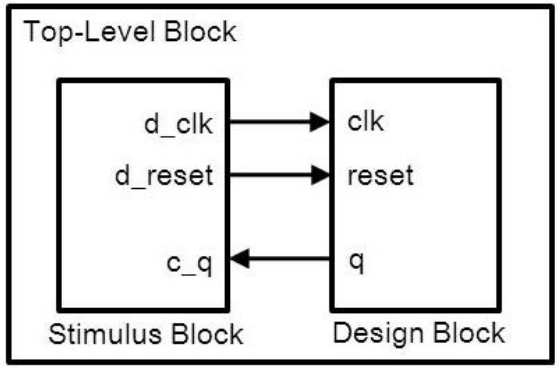
Simulation Output

```
  0 Output q =  0
 20 Output q =  1
 30 Output q =  2
 40 Output q =  3
 50 Output q =  4
 60 Output q =  5
 70 Output q =  6
 80 Output q =  7
 90 Output q =  8
100 Output q =  9
110 Output q = 10
120 Output q = 11
130 Output q = 12
140 Output q = 13
150 Output q = 14
160 Output q = 15
170 Output q =  0
180 Output q =  1
190 Output q =  2
195 Output q =  0
210 Output q =  1
220 Output q =  2
```

Testbench

2. Stimulus and Design Blocks instantiated in a Dummy Top-Level Module:

- In the second style of applying stimulus, both the stimulus and design blocks are instantiated in to a top-level dummy module.
- The function of top-level block is simply to instantiate the design and stimulus blocks.
- The stimulus block interacts with the design block only through the interface.



Summary

- Introduction
- Hardware Description Languages.
- Design Methodologies.
- Lexical Conventions
  - Data types.
  - Numbers.
  - Operators.
- Testbench